Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Single Sensor Multi-Tenant Log Ingestion

For multi-tenant deployments, you typically configure one Modular Sensor to ingest logs per tenant. Optionally, you can consolidate this task to a single Modular Sensor for multiple tenants. The details below identify which vendors' log files are currently supported for this feature, along with the associated configuration steps.

- Enable Multi-Tenant Log Ingestion
- Stellar Cyber JSON Format for Multi-Tenant Parsing
- Configure Exium
- Methods to Add Tenant IDs
    - Method 1 – Add the Tenant ID in JSON Payloads
    - Method 2 – Add the Tenant ID in the HTTP Header
    - Method 3 – Add the Tenant ID Using an NGINX Reverse Proxy

Enable Multi-Tenant Log Ingestion

Multi-Tenant Log Ingestion is off by default and must be enabled in the Modular Sensor profile for Stellar Cyber to recognize the tenant ID included in logs. If this isn't enabled, Stellar Cyber uses the tenant ID of the sensor instead.

1. To enable it, navigate to **System | DATA SOURCE MANAGEMENT | Sensors | Sensor Profiles** and then select the **Edit** icon for a Modular Sensor profile.
2. Expand the **Log Forwarder** section and toggle on **Multi-Tenant Log Ingestion**.

   After it's enabled, you can send data from the following vendors:

Vendor	Product	Format	Port	Tenant ID Handling
Infocyte	HUNT	CEF	5143 or 5870	The source includes a built-in tenant ID field.
Blackberry	Cylance	KVP	5177	The source includes a built-in tenant ID field.

Vendor	Product	Format	Port	Tenant ID Handling
Exium	–	HTTP JSON	5200	The tenant ID is configured in the Exium Access Console.
Palo Alto Networks	Firewall	Syslog, CEF, LEEF	5564*	The tenant ID is added through JSON.
SonicWall	Firewall	Syslog, CEF	5564*	The tenant ID is added through JSON.
Cisco	Meraki	Syslog	5564*	The tenant ID is added through JSON.
Fortinet	Fortigate	KVP Syslog	5564*	The tenant ID is added through JSON.

* These are supported for use on port 5564 if submitted in the Stellar Cyber JSON format indicated below

Stellar Cyber JSON Format for Multi-Tenant Parsing

To learn more about JSON, see these websites…

JavaScript Object Notation (JSON) is a lightweight data interchange format that's easy for humans to read and write, and easy for machines to parse and generate. It consists of two primary structures: a collection of name/value pairs and an ordered list of values. If you're interested in learning more about JSON, here are some useful resources:

- For an introduction to JSON, see Introducing JSON.
- For a basic reference, see JSON.
- For a JSON error correction and validation tool, see JSONLint Validator and Formatter.

An example Stellar Cyber JSON record is shown here (data is truncated)

```
{
  "stellar": {
    "tenantid": "12345678",
    "msg_origin": {
      "vendor": "palo_alto_networks",
      "product": "firewall",
      "format": "syslog",
      "srcip": "192.168.100.74"
    }
  },
  "parser_raw_msg":"<134>Nov 11 2020 21:55:21 ..."
}
```

This required format consists of four parts:

- `stellar`: This required field is used to specify that the file is in Stellar Cyber format.
- `tenantid`: Specify the tenant ID. If this value is not present, the Tenant ID associated with the sensor is used.

- `msg_origin`: This wrapper label is required for parsing on the multi-tenant port. Use the following table for guidance with the relevant fields.

vendor	product	format
palo_alto_networks	firewall	syslog
		cef
		leef
sonicwall	firewall	syslog
		cef
cisco	meraki	syslog
fortinet	fortigate	syslog

    - `vendor` and `product`: Both of these values in the `msg_origin` block are required. They are used to identify which Stellar Cyber parser will handle the original logs.
    - `format`: If this value is not present, syslog is used.
    - `srcip`: This value should be the IP address of the original log source. It is optional, but including it results in better log ingestion statistics.
- `parser_raw_msg`: If this required content is missing, the record is discarded.

> Because the raw data is wrapped in double quotes, ensure that the pre-processor you use to create this record injects an escape before all double quotes in the original log file.

The field labels are illustrated here:

```
{
  "stellar": {
    "tenantid": " ",
    "msg_origin": {
      "vendor": " ",
      "product": " ",
      "format": " ",
      "srcip": " "
    }
  },
  "parser_raw_msg":"  "
}
```

Configure Exium

In v5.1.1, Stellar Cyber released a built-in Exium parser that supports multi-tenancy. To set up Exium to provide Stellar Cyber with the tenant ID for the logs it sends, log in to the Exium Admin Console and configure the tenant ID for each tenant and the URL of the Exium parser on the

Stellar Cyber sensor to which Exium tenants send logs (`https://<sensor_ip_addr>:5200/httpjson`). With Multi-Tenant Log Ingestion enabled on the Modular Sensor profile, the Stellar Cyber Platform associates the logs it ingests with the various Exium tenants that send them. For configuration instructions from Exium, see StellarCyber SIEM Integration.

Methods to Add Tenant IDs

While some third-party products like Exium have added settings to their systems to integrate with Stellar Cyber and provide tenant IDs with the logs they send, others have not. However, these systems can still support multi-tenancy if you configure the devices that send logs to the multi-tenant parser to add their tenant ID to either JSON payloads or custom HTTP headers before they send their logs. Stellar Cyber v5.2.0 and later supports both methods. The option you choose depends on the capabilities of the devices sending the logs.

Method 1 – Add the Tenant ID in JSON Payloads

If your devices let you configure the log-forwarder that runs on them to include a tenant ID in the JSON payloads of the logs they send, then configure them to do that. The following is an example log message that includes the tenant ID in the JSON payload.

```
curl -X POST -d '{"tenantid": "67379250", "Event Type": "Device", "Event Name": "SystemSecurity", "Device Name": "SGI-TWR-B00XQS3", "Agent Version": "3.1.1001.15", "IP Address": "10.1.1.100", "MAC Address": "112233BB12AB", "Logged On Users": "ACME\jsmith", "OS": "Microsoft Windows 10", "Kernel Version": "10.0.19042", "Optics Version": "3.0.1178.0", "Zone Names": "Acme Workstations"}' 10.1.1.25:5200/httpjson
```

If your devices don't let you modify the JSON payload like this, consider using custom HTTP headers instead.

Method 2 – Add the Tenant ID in the HTTP Header

If your devices let you add a custom HTTP header with a tenant ID to their HTTP output, then configure the log-forwarder on these devices to include the tenant ID in the custom HTTP header when they send logs to the parser. The following is an example log message that includes the tenant ID in the HTTP header.

```
curl -X POST -H "Stellar-TenantID: 67379250" -d '{"Event Type": "Device", "Event Name": "SystemSecurity", "Device Name": "SGI-TWR-B00XQS3", "Agent Version": "3.1.1001.15", "IP Address": "10.1.1.100", "MAC Address": "112233BB12AB", "Logged On Users": "ACME\jsmith", "OS": "Microsoft Windows 10", "Kernel Version": "10.0.19042", "Optics Version": "3.0.1178.0", "Zone Names": "Acme Workstations"}' 10.1.1.25:5200/httpjson
```

Method 3 – Add the Tenant ID Using an NGINX Reverse Proxy

If your devices don't let you add an HTTP custom header to logs, you can configure them to forward logs to an NGINX server. Configure that to add a custom HTTP header with a tenant ID based on information identifying the source of the logs it receives, such as the source IP address, before forwarding them to the parser on the Modular Sensor.

The following is a configuration example for an NGINX server to add a custom HTTP header with the parser URL and tenant ID to logs based on source IP address.

```
server {
        listen 80;
        set $stellar_tenantid "default_tenantid";
        if ($remote_addr = "10.1.1.1") {
                set $stellar_tenantid "tenantid_a";
        }
        if ($remote_addr = "10.2.1.1") {
                set $stellar_tenantid "tenantid_b";
        }
        location / {
                proxy_pass http://<stellar-device-sensor>:<http-parser-port>;
                proxy_set_header Stellar-TenantID $stellar_tenantid;
        }
}
```

When a log reaches the NGINX server from 10.1.1.1, the server sets the Stellar-TenantID header as `tenantid_a` and forwards it to the specified URL and port number of the multi-tenant parser. When a log reaches the NGINX server from 10.2.1.1, it sets the Stellar-TenantID header as `tenantid_b` and forwards it to the same URL and port number of the parser. Finally, if a log reaches the NGINX server from any other IP address, it sets the Stellar-TenantID header as `default_tenantid` and forwards it to the same destination as the others. In this way, Stellar Cyber can distinguish logs from different tenants using a single sensor with a multi-tenant parser.

© 2026 Stellar Cyber . All rights reserved.